

# Tacti-AI Web Project

---

## Complete Documentation

A comprehensive guide to project architecture, setup, deployment, and best practices

Generated: July 3, 2026

Version: 1.0

Project Type: React + TypeScript + Vite SPA

---



## Table of Contents

1. Project Overview
  2. Technology Stack & Architecture
  3. Project Structure & File Organization
  4. File-by-File Breakdown
  5. Local Setup & Installation
  6. Development Workflow
  7. Building & Testing
  8. Deployment Guide
  9. GoDaddy Hosting Instructions
  10. Troubleshooting & Best Practices
-

# 1. Project Overview

---

## About Tacti-AI Web

The Tacti-AI Web project is a modern, responsive single-page application (SPA) built with React and TypeScript. It serves as the company's main website, showcasing capabilities, projects, research, and career opportunities.

### Project Features

- ✓ Responsive Design (Mobile, Tablet, Desktop)
- ✓ Server-Side Rendering Support
- ✓ Modern UI Components (shadcn/ui)
- ✓ Type-Safe Development (TypeScript)
- ✓ Performance Optimized (Vite)
- ✓ SEO Friendly
- ✓ Multiple Pages & Routes
- ✓ Professional Styling (Tailwind CSS)
- ✓ Toast Notifications
- ✓ Form Handling & Validation

### Live URLs

**Current Deployment:** <https://militros.onrender.com>

**Project Template:** Based on Lovable (formerly known as Relume)

---

## 2. Technology Stack & Architecture

### Core Technologies

| Technology           | Version | Purpose                              |
|----------------------|---------|--------------------------------------|
| React                | 18.x    | UI Library & Component Framework     |
| TypeScript           | 5.x     | Type Safety & Development Experience |
| Vite                 | Latest  | Build Tool & Dev Server              |
| Tailwind CSS         | 3.x     | Utility-First Styling                |
| shadcn/ui            | Latest  | Pre-built UI Components              |
| React Router         | 6.x     | Client-Side Routing                  |
| TanStack React Query | 5.x     | Server State Management              |
| React Hook Form      | 7.x     | Form Handling & Validation           |
| Radix UI             | Latest  | Accessible Component Primitives      |
| Bun                  | Latest  | JavaScript Runtime & Package Manager |

### Development Tools

- **ESLint:** Code quality & linting
- **Vitest:** Unit testing framework
- **PostCSS:** CSS processing
- **Express.js:** Server runtime for production
- **Node.js:** JavaScript runtime

# Architecture Overview

The application follows a **component-based architecture** with:

- **Single Page Application (SPA):** Client-side routing
  - **Component Composition:** Modular, reusable components
  - **Separation of Concerns:** Pages, Components, UI, Hooks, Utils
  - **Type Safety:** Full TypeScript support
  - **Styling:** Tailwind CSS with CSS Modules where needed
  - **State Management:** React hooks + React Query for server state
-

## 3. Project Structure & File Organization

---

### Directory Tree

```

root/ ├── src/ # Application source code | ├── components/ # React components | | ├── ui/ #
Reusable UI components (shadcn) | | ├── AboutSection.tsx | | ├── HeroSection.tsx | | ├──
Navbar.tsx | | ├── Footer.tsx | | └── ... other sections | ├── pages/ # Page components
(routes) | | ├── Index.tsx # Home page | | ├── Contact.tsx # Contact page | | ├── Careers.tsx
# Careers page | | ├── Login.tsx # Login page | | └── NotFound.tsx # 404 page | ├── hooks/ #
Custom React hooks | | ├── use-mobile.tsx # Mobile detection hook | | └── use-toast.ts #
Toast notification hook | ├── lib/ # Utility functions | | └── utils.ts # Helper functions |
└── assets/ # Static assets | ├── App.tsx # Main app component | ├── main.tsx # Entry point |
└── index.css # Global styles | └── App.css # App styles | ├── public/ # Static public files |
└── images/ # Image assets | ├── videos/ # Video assets | ├── clients/ # Client logos | ├──
achievements/ # Achievement images | ├── robots.txt # SEO robot rules | └── _redirects #
Redirect rules | ├── dist/ # Build output (generated) | ├── package.json # Project dependencies
└── vite.config.ts # Vite configuration | ├── tsconfig.json # TypeScript configuration | ├──
tailwind.config.ts # Tailwind CSS configuration | ├── postcss.config.js # PostCSS configuration
└── render.yaml # Render.com deployment config | ├── server.js # Express server for production
└── eslint.config.js # ESLint configuration | ├── index.html # HTML entry point | └── README.md #
Project README

```

---

## 4. File-by-File Breakdown

### Configuration Files

#### **package.json**

Defines project metadata, dependencies, and npm scripts. Contains all required packages and build commands.

*Key Scripts:*

- `npm run dev` - Start development server
- `npm run build` - Build for production
- `npm start` - Run production server
- `npm test` - Run tests
- `npm run lint` - Lint code

#### **vite.config.ts**

Vite build configuration. Includes server settings, plugin configuration, and path aliases.

*Important Settings:*

- Port: 3000 (development)
- React SWC plugin for fast transpilation
- Path alias: `@` → `./src`
- Allowed hosts for Render.com deployment

#### **tsconfig.json**

TypeScript compiler configuration. Ensures consistent type checking across the project.

#### **tailwind.config.ts**

Tailwind CSS configuration. Defines custom theme, colors, fonts, and component customizations.

*Customizations:*

- Custom fonts (Inter, Playfair Display)
- Dark mode support
- Custom color palette with CSS variables

### **postcss.config.js**

PostCSS configuration for processing CSS with Tailwind and Autoprefixer.

### **eslint.config.js**

ESLint configuration for code quality and consistency checks.

### **render.yaml**

Render.com deployment configuration. Specifies build commands, start command, and SPA routing rules.

### **server.js**

Express.js server for production. Serves static files from `dist/` directory and handles SPA routing.

## Source Code Files

### **src/App.tsx**

Root component. Sets up routing, providers (React Query, Tooltips), and global UI components (Toasters).

*Routes Defined:*

-  - Home page (Index)

- `/careers` - Careers page
- `/contact` - Contact page
- `/login` - Login page
- `*` - 404 Not Found page

#### **src/main.tsx**

Application entry point. Mounts React app to DOM and applies global styles.

## Page Components

#### **src/pages/Index.tsx**

Home page. Displays hero section, mission, capabilities, projects, research, news, testimonials, and career sections.

#### **src/pages/Contact.tsx**

Contact page. Contains contact form and company information.

#### **src/pages/Careers.tsx**

Careers page. Job listings and career information.

#### **src/pages/Login.tsx**

Login page. Authentication interface.

#### **src/pages/NotFound.tsx**

404 page. Displays when route is not found.

## Component Files

 **src/components/HeroSection.tsx**

Hero/banner section with main call-to-action and branding.

 **src/components/Navbar.tsx**

Navigation bar with responsive menu and brand logo.

 **src/components/NavLink.tsx**

Navigation link component with active state styling.

 **src/components/[\*]Section.tsx**

*Multiple section components:*

- AboutSection - Company information
- MissionSection - Mission statement
- CapabilitiesSection - Key capabilities
- ProjectsSection - Portfolio projects
- ResearchSection - Research & publications
- DomainsSection - Industry domains
- NewsSection - News & updates
- TestimonialsSection - Client testimonials
- SecuritySection - Security information
- TechnologyShowcase - Tech stack
- CollaboratorsSection - Partners
- CareersSection - Career opportunities
- Footer - Footer with links & info

 **src/components/ui/**

Pre-built UI components from shadcn/ui library. These are composable, accessible components.

*Includes:* Button, Card, Dialog, Form, Input, Select, Table, Tabs, Accordion, Carousel, Chart, and many more.

## Hooks & Utils

### **src/hooks/use-mobile.tsx**

Custom hook for detecting mobile viewport.

### **src/hooks/use-toast.ts**

Custom hook for displaying toast notifications.

### **src/lib/utils.ts**

Utility functions including class name merging with Clsx.

## Styling Files

### **src/index.css**

Global styles including Tailwind imports and CSS variable definitions for the theme.

### **src/App.css**

App-specific styles.

# 5. Local Setup & Installation

## Prerequisites

Before starting, ensure you have the following installed:

- **Node.js** (v16 or higher) - <https://nodejs.org>
- **Bun** (optional but recommended) - <https://bun.sh>
- **Git** - <https://git-scm.com>
- **npm or yarn** - Usually comes with Node.js

## Installation Steps

### Step 1: Clone the Repository

```
git clone <YOUR_REPOSITORY_URL>
cd tacti-ai-web-updated
```

### Step 2: Install Dependencies

```
# Using npm
npm install

# OR using Bun
bun install
```

### Step 3: Verify Installation

```
npm run lint
```

This should run without errors if installation was successful.

## Environment Setup

Check if any environment variables are needed. Create a `.env` file if required (not currently needed for basic setup).

 **Note:** The project doesn't require any API keys or environment variables for basic local development.

---

## 6. Development Workflow

### Starting Development Server

#### Start the Development Server

```
npm run dev
```

Output: The server will start on `http://localhost:3000`

The development server includes:

- ✓ Hot Module Replacement (HMR) - Changes reflect instantly
- ✓ Error overlay - Shows compilation errors in browser
- ✓ Auto-refresh - Page reloads on file changes

### Making Changes

#### Editing Components

Components are located in `src/components/`

1. Open a component file (e.g., `src/components/HeroSection.tsx`)
2. Make your changes
3. Save the file (Ctrl+S)
4. Browser automatically updates

#### Adding New Components

```
// src/components/MyNewComponent.tsx
import React from 'react';

export const MyNewComponent = () => {
  return (
    <div>
      <h1>My New Component</h1>
    </div>
  )
}
```

```
    </div>
  );
};

export default MyNewComponent;
```

## Adding New Pages

Create a new file in `src/pages/`, then add route in `src/App.tsx`:

```
// In src/App.tsx
import MyNewPage from "../pages/MyNewPage";

const App = () => (
  // ...
  <Routes>
    <Route path="/" element={<Index />} />
    <Route path="/my-page" element={<MyNewPage />} />
    // ... other routes
  </Routes>
  // ...
);
```

## Styling with Tailwind CSS

Use Tailwind utility classes directly in your JSX:

```
<div className="flex items-center justify-between p-4 bg-blue-500 rounded-lg">
  <h2 className="text-2xl font-bold text-white">Title</h2>
  <button className="px-4 py-2 bg-white text-blue-500 rounded hover:bg-gray-100">
    Click Me
  </button>
</div>
```

## Using UI Components

Import and use pre-built components from `shadcn/ui`:

```
import { Button } from "@components/ui/button";
import { Card, CardContent } from "@components/ui/card";

export const MyComponent = () => {
  return (
    <Card>
      <CardContent>
        <Button>Click Me</Button>
      </CardContent>
    </Card>
  );
};
```

```
        </CardContent>
      </Card>
    );
  };
};
```

## Form Handling

Use React Hook Form for form management:

```
import { useForm } from "react-hook-form";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";

export const ContactForm = () => {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = (data) => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <Input {...register("email", { required: true })} />
      {errors.email && <span>Email is required</span>}
      <Button type="submit">Submit</Button>
    </form>
  );
};
```

## Verifying Changes

### Visual Testing

1. Open browser to `http://localhost:3000`
2. Navigate to affected pages
3. Test on mobile viewport (F12 → Toggle device toolbar)

### Browser DevTools

- Use React Developer Tools extension to inspect components
- Check Console for warnings/errors
- Test responsive design at different breakpoints

### Responsive Design Testing

## Test breakpoints defined in Tailwind:

- `sm` - 640px
  - `md` - 768px
  - `lg` - 1024px
  - `xl` - 1280px
  - `2xl` - 1536px
-

## 7. Building & Testing

### Building for Production

#### Build Production Bundle

```
npm run build
```

This command:

- Installs latest dependencies with Bun
- Compiles TypeScript to JavaScript
- Bundles all code with Vite
- Minifies and optimizes assets
- Outputs to `dist/` folder

 **Build Output:** The `dist/` folder contains all files needed for production deployment.

### Preview Production Build

Before deploying, preview the production build locally:

```
npm run build    # Build first
npm run preview  # Preview the build
```

This serves the optimized build from `dist/` at `http://localhost:4173`

### Running Tests

#### Run All Tests (Once)

```
npm test
```

### Run Tests in Watch Mode

```
npm run test:watch
```

Tests are configured with Vitest and located in `src/test/`

## Linting Code

### Check Code Quality

```
npm run lint
```

ESLint checks your code for style inconsistencies and potential errors.

## Build Troubleshooting

**⚠ Build Failed:** Clear node\_modules and reinstall

```
rm -rf node_modules package-lock.json  
npm install
```

**⚠ TypeScript Errors:** Check tsconfig.json and ensure all imports are correct

```
npx tsc --noEmit
```

## 8. Deployment Guide

### Current Deployment (Render.com)

The project is currently configured to deploy on [Render.com](https://render.com).

#### How It Works

1. Push code to Git repository
2. Render.com automatically detects `render.yaml`
3. Runs build command: `npm install && npm run build`
4. Starts server with: `npm start`
5. Serves from `dist/` directory

#### Deployment Configuration (render.yaml)

```
services:
  - type: web
    name: militros
    env: node
    plan: free
    buildCommand: npm install && npm run build
    startCommand: npm start
    staticPublishPath: dist

rewrites:
  - source: /*
    destination: /index.html
    type: rewrite
```

### Deploying to Render.com

#### Step 1: Create Render Account

Go to <https://render.com> and sign up

#### Step 2: Connect GitHub Repository

1. Click "New" → "Web Service"
2. Select "Deploy an existing repository"
3. Connect your GitHub account
4. Select the repository

### Step 3: Configure Service

1. Name: militros (or your preferred name)
2. Environment: Node
3. Build Command: `npm install && npm run build`
4. Start Command: `npm start`

### Step 4: Deploy

Click "Create Web Service" and Render will automatically deploy your application

## Environment Variables (if needed)

To add environment variables in Render.com:

1. Go to your service dashboard
2. Click "Environment" tab
3. Add environment variables as key-value pairs
4. Trigger a redeploy

## Custom Domain on Render.com

1. Go to your service settings
2. Scroll to "Custom Domain"
3. Enter your domain
4. Update DNS records (Render provides instructions)



# 9. GoDaddy Hosting Instructions

## Overview

GoDaddy offers multiple hosting options. For this SPA, we recommend using GoDaddy's Linux Hosting or Managed WordPress with static file serving.

## Option 1: GoDaddy Linux Hosting (Recommended)

### Step 1: Purchase Hosting Plan

1. Go to [GoDaddy.com](https://GoDaddy.com)
2. Click "Hosting" → "Web Hosting"
3. Choose a Linux Hosting plan
4. Complete purchase

### Step 2: Access GoDaddy Control Panel

1. Log in to [GoDaddy Account](#)
2. Find your hosting account
3. Click "Launch" or "Manage" button
4. You'll be taken to cPanel

### Step 3: Prepare Your Application

1. Build the project locally:

```
npm run build
```

2. This creates a `dist/` folder with all production files
3. You'll upload the contents of this folder to your hosting

## Step 4: Create .htaccess File (SPA Routing)

For React Router to work on GoDaddy, create a `.htaccess` file in your root directory:

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /

  # Don't rewrite actual files or directories
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d

  # Rewrite all other requests to index.html
  RewriteRule ^ index.html [QSA,L]
</IfModule>
```

Save this as `.htaccess` (note the leading dot)

## Step 5: Upload Files via cPanel File Manager

1. In cPanel, find "File Manager"
2. Navigate to `public_html` folder
3. Click "Upload" button
4. Select all files from your `dist/` folder
5. Also upload the `.htaccess` file
6. Wait for upload to complete

## Step 6: Upload via FTP (Alternative)

If File Manager is slow, use FTP:

1. In cPanel, find "FTP Accounts"
2. Create a new FTP account or note existing credentials
3. Download an FTP client (Filezilla, WinSCP, or Cyberduck)
4. Connect using provided FTP credentials
5. Navigate to `public_html`
6. Upload all files from `dist/` folder

## 7. Upload `.htaccess` file

### Step 7: Connect Domain

1. If domain is registered at GoDaddy, it's already pointed to your hosting
2. If domain is elsewhere, update its DNS records to point to GoDaddy nameservers
3. Check DNS propagation at [whatsmydns.net](https://whatsmydns.net)
4. Usually takes 24-48 hours to fully propagate

### Step 8: Test Your Website

1. Visit your domain in a browser
2. Test all routes (/careers, /contact, /login, etc.)
3. Test on mobile devices
4. Check browser console for errors (F12)

## Option 2: GoDaddy cPanel with Node.js Application

Some GoDaddy plans support Node.js applications directly:

### Prerequisites

- GoDaddy hosting plan that supports Node.js
- Access to cPanel and Terminal

### Deployment Steps

1. Access cPanel Terminal (Advanced users)
2. Clone your repository:

```
cd ~/public_html
git clone <YOUR_REPO_URL>
```

3. Install dependencies:

```
npm install
```

#### 4. Build the application:

```
npm run build
```

#### 5. Create Node.js application in cPanel:

1. Find "Setup Node.js App" in cPanel
2. Create new application
3. Point to your project directory
4. Set application startup file to `server.js`

#### 6. Restart the application

## Troubleshooting GoDaddy Deployment

### ⚠ Blank Page or 404 Error

- Ensure `.htaccess` file is uploaded correctly
- Check that all files from `dist/` are in `public_html`
- Clear browser cache (Ctrl+F5)
- Check GoDaddy server error logs in cPanel

### ⚠ Routes Not Working

- Verify `.htaccess` file is in root of `public_html`
- Ensure `mod_rewrite` is enabled on server
- Check file permissions (should be 644 for files)

### ⚠ Slow Performance

- Enable gzip compression in cPanel
- Use GoDaddy CDN if available

- Minimize image sizes
- Consider upgrading hosting plan

### **SSL Certificate Issues**

- GoDaddy offers free SSL with most plans
  - In cPanel, find "SSL/TLS Status"
  - Install AutoSSL certificate
  - Wait for certificate to install (can take 24 hours)
-

# 10. Troubleshooting & Best Practices

---

## Common Issues & Solutions

### Issue: "npm install" fails

#### Solution:

```
rm -rf node_modules
rm package-lock.json
npm install
```

### Issue: Port 3000 already in use

#### Solution:

```
# Kill process on port 3000
lsof -ti:3000 | xargs kill -9

# Or use different port
npm run dev -- --port 3001
```

### Issue: TypeScript compilation errors

#### Solution:

1. Check for import errors (correct paths)
2. Ensure all types are imported
3. Run: `npx tsc --noEmit` for detailed errors

### Issue: Styling not loading (Tailwind CSS)

**Solution:**

1. Clear build cache: `rm -rf dist`
2. Restart dev server: `npm run dev`
3. Check tailwind.config.ts content paths are correct

**Issue: Hot reload not working****Solution:**

1. Check HMR settings in vite.config.ts
2. Try full page refresh (Ctrl+Shift+R)
3. Restart dev server

**Issue: 404 errors after deployment****Solution:**

1. Verify `.htaccess` is in root directory (GoDaddy)
2. Check SPA routing configuration
3. Ensure index.html is in dist folder
4. Test with full URL including domain

## Best Practices

### 1. Code Organization

- Keep components focused and single-responsibility
- Use descriptive file and component names
- Group related components in subdirectories
- Separate presentation from business logic

### 2. Naming Conventions

- **Components:** PascalCase (MyComponent.tsx)

- **Hooks:** camelCase starting with 'use' (useMyHook.ts)
- **Files:** Match component names (MyComponent.tsx)
- **Variables:** camelCase (myVariable)
- **Constants:** UPPER\_SNAKE\_CASE (MY\_CONSTANT)

### 3. Performance Optimization

- **Code Splitting:** Use React.lazy for route-based splitting
- **Image Optimization:** Compress images before adding
- **Memoization:** Use React.memo for expensive components
- **Lazy Loading:** Defer loading of non-critical resources
- **Bundle Analysis:** Monitor build size regularly

### 4. Type Safety

- Always use TypeScript types for props
- Define interfaces for component props
- Avoid using `any` type
- Enable strict mode in tsconfig.json

### 5. Git Workflow

- Create feature branches for new work
- Write meaningful commit messages
- Keep commits small and focused
- Review code before merging to main

### 6. Testing

- Write unit tests for utility functions
- Test component rendering and interactions
- Maintain > 80% code coverage
- Run tests before each deployment

### 7. Accessibility

- Use semantic HTML (buttons, links, etc.)

- Add alt text to images
- Ensure keyboard navigation works
- Test with screen readers
- Maintain good color contrast

## 8. Security

- Never commit sensitive data (.env files)
- Use HTTPS for all connections
- Validate and sanitize user input
- Keep dependencies updated
- Regular security audits: `npm audit`

## 9. SEO Optimization

- Use semantic HTML tags
- Add meta tags and descriptions
- Create sitemap.xml
- Use descriptive URLs
- Add Open Graph meta tags for social sharing

## 10. Documentation

- Keep README.md up to date
- Document complex functions
- Add JSDoc comments for public APIs
- Maintain this project documentation

# Development Checklist

Before deploying, verify:

- |                                                                           |                                                                              |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------|
| • <input type="checkbox"/> All changes are committed                      | • <input type="checkbox"/> Build succeeds: <code>npm run build</code>        |
| • <input type="checkbox"/> Code passes linting: <code>npm run lint</code> | • <input type="checkbox"/> No build warnings                                 |
| • <input type="checkbox"/> Tests pass: <code>npm test</code>              | • <input type="checkbox"/> Preview build works: <code>npm run preview</code> |
| • <input type="checkbox"/> No console errors in dev mode                  | • <input type="checkbox"/> Tested on mobile viewport                         |

- ☐ All routes work correctly
  - ☐ Forms submit properly
  - ☐ Images load correctly
  - ☐ Performance is acceptable
-

# Appendix: Quick Reference

## Useful Commands

| Command                         | Purpose                   |
|---------------------------------|---------------------------|
| <code>npm run dev</code>        | Start development server  |
| <code>npm run build</code>      | Build for production      |
| <code>npm run preview</code>    | Preview production build  |
| <code>npm start</code>          | Run production server     |
| <code>npm test</code>           | Run tests once            |
| <code>npm run test:watch</code> | Run tests in watch mode   |
| <code>npm run lint</code>       | Check code quality        |
| <code>npm install</code>        | Install dependencies      |
| <code>bun install</code>        | Install with Bun (faster) |

## Key File Paths

| Path                         | Purpose                  |
|------------------------------|--------------------------|
| <code>src/</code>            | All source code          |
| <code>src/pages/</code>      | Page components (routes) |
| <code>src/components/</code> | Reusable components      |
| <code>src/hooks/</code>      | Custom React hooks       |

| Path     | Purpose                  |
|----------|--------------------------|
| src/lib/ | Utility functions        |
| public/  | Static assets            |
| dist/    | Build output (generated) |

## Resources & Links

- **React Documentation:** <https://react.dev>
- **TypeScript Documentation:** <https://www.typescriptlang.org>
- **Vite Documentation:** <https://vitejs.dev>
- **Tailwind CSS:** <https://tailwindcss.com>
- **shadcn/ui:** <https://ui.shadcn.com>
- **React Router:** <https://reactrouter.com>
- **GoDaddy Support:** <https://www.godaddy.com/help>
- **Render.com Documentation:** <https://render.com/docs>

## Support & Contact

For issues or questions:

- Check the troubleshooting section above
- Review existing documentation
- Search GitHub issues in related projects
- Contact your development team

### Tacti-AI Web Project Documentation

Version 1.0 | Generated July 3, 2026

This documentation is a comprehensive guide for developers and deployment teams.